→ The Ideal.

⇒ Uniform control flow.

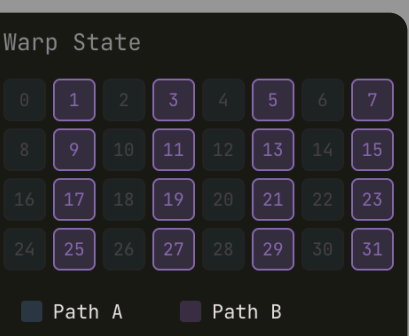
```
// no branches
// all threads execute
     the same op.
out[i] = a[i] + b[i];
```

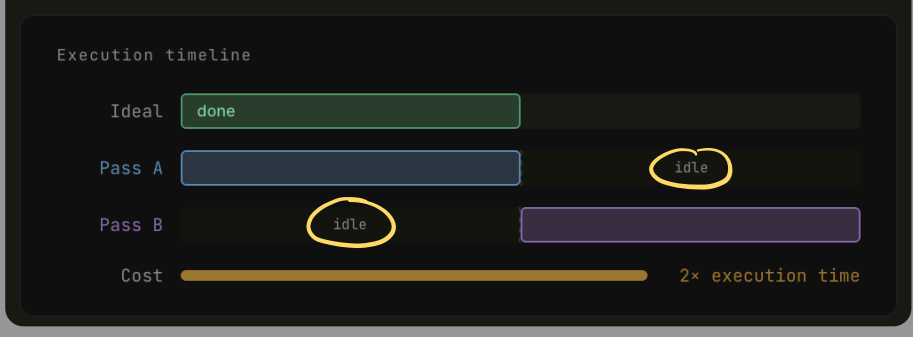
> all 32 threads, in parallel, execute the same instruction
> - NO divergence -

→ What causes Divergence.

⇒ CONDITIONAL Branches.

```
// if/else: threads diverge
if (x[tid] > threshold) {
    result[tid] = compute_A();
} else {
    result[tid] = compute_B();
}
```

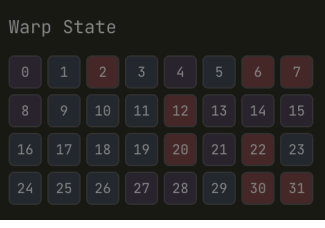




> The hardware serializes execution
> Two passes required ⇒ 50% eff.

⇒ Variable Loops.

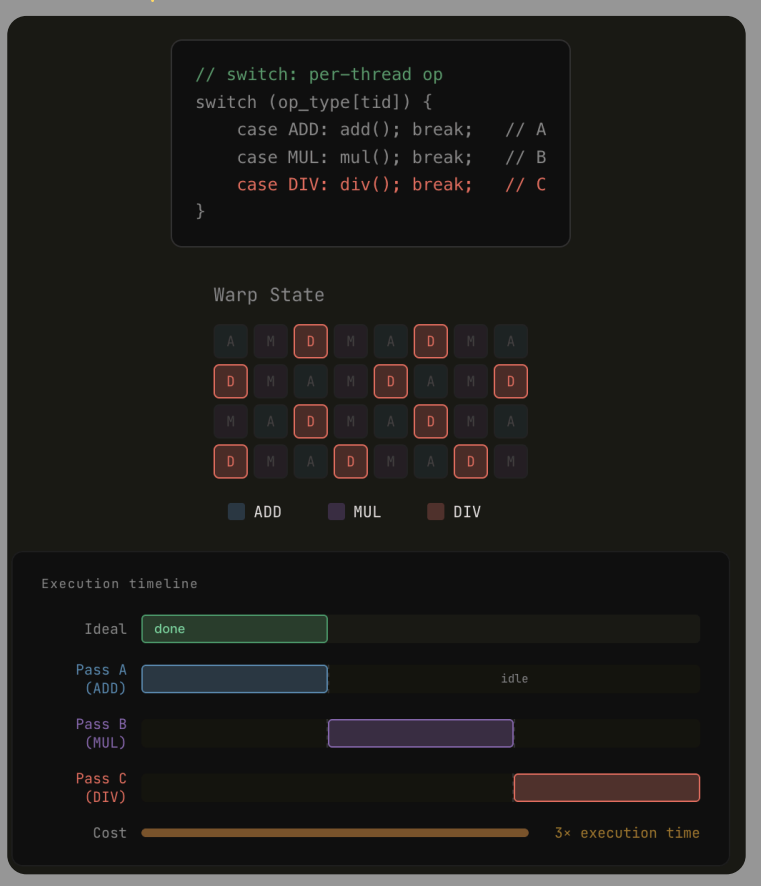
```
// loop count varies by thread
int n = work_count[tid];
for (int i = 0; i < n; i++) {
    result[tid] += compute(i);
}
```



> Threads with fewer iterations finish early but must wait idle
The warp cannot complete until the slowest thread finishes

⇒ Dynamic functions.

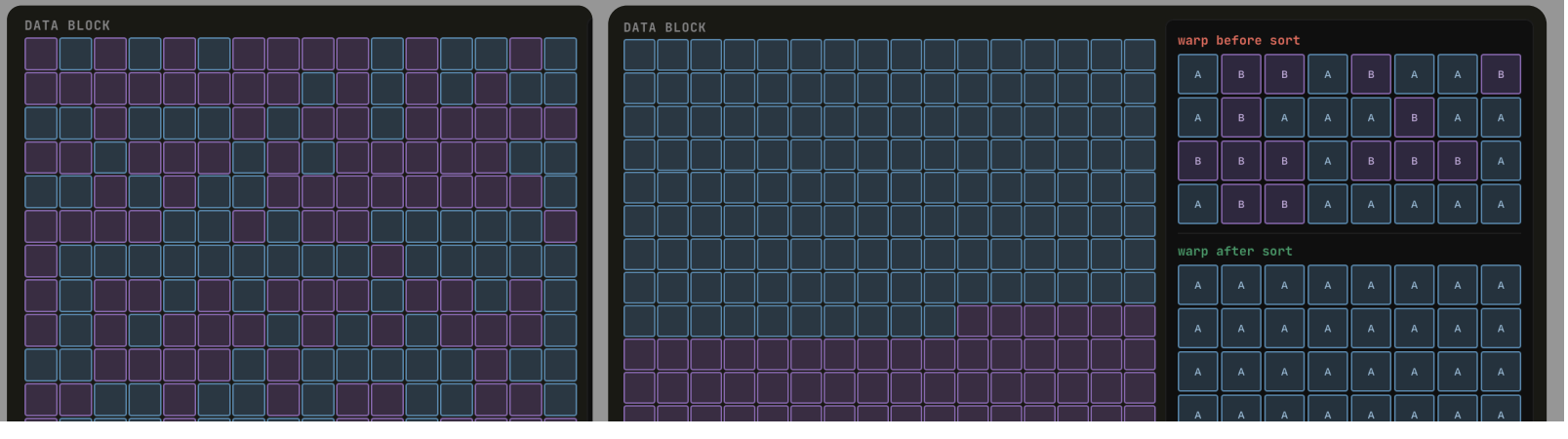
```
// switch: per-thread op
switch (op_type[tid]) {
    case ADD: add(); break;   // A
    case MUL: mul(); break;   // B
    case DIV: div(); break;   // C
}
```

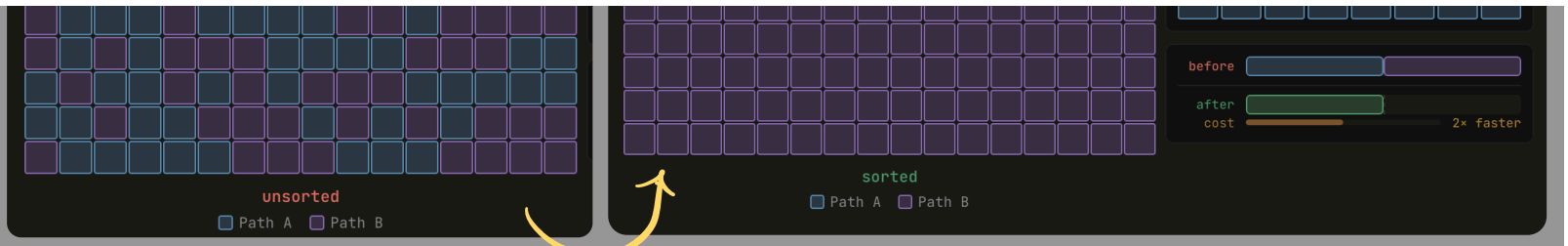


> 3 unique function calls
force 3 serial execution phases
Efficiency drops to
~33%

→ How to fix it?

⇒ Data Sorting.

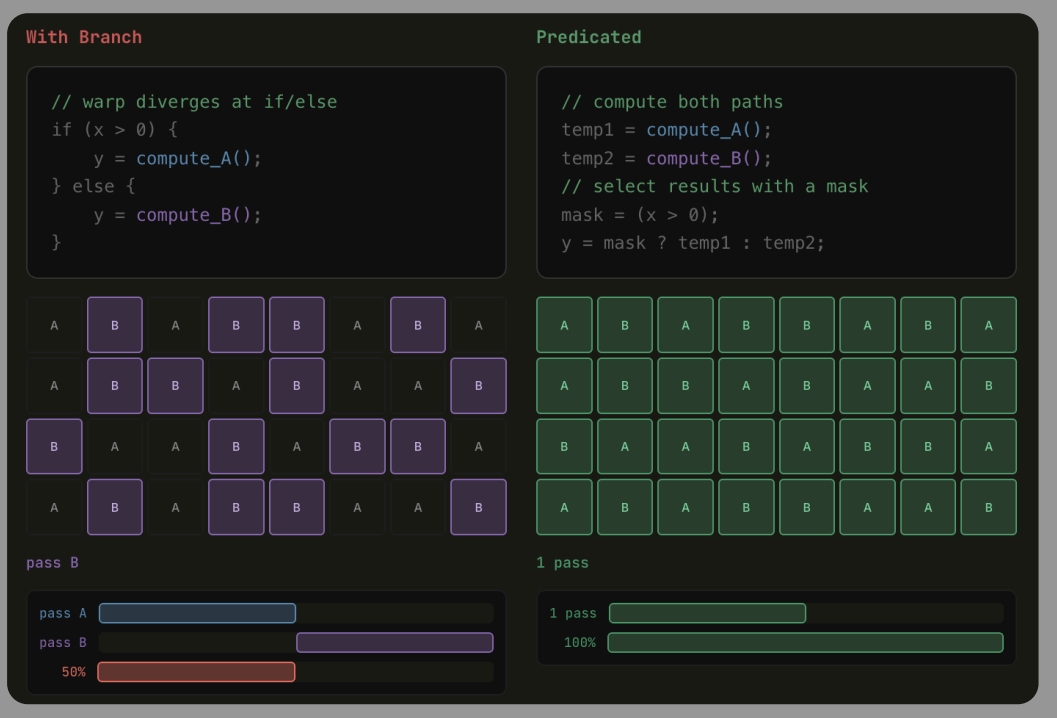




sorting by path

> Sort work items by execution path before launching
Adjacent threads then follow the same path;
meaning
coherent warps and no divergence

=> Predication

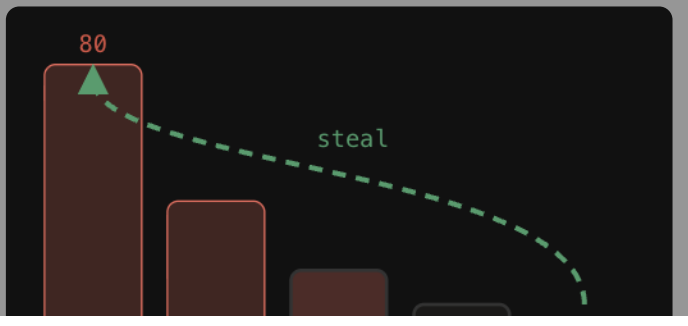


> execute both paths simultaneously
then mask the result
No branch => no divergence

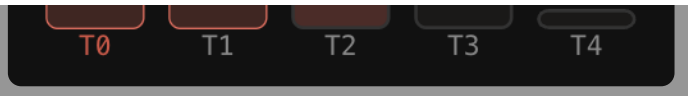
=> Load Balancing

→ Redistribute work so no thread is straggler
Uniform load == no thread sits idle/waiting

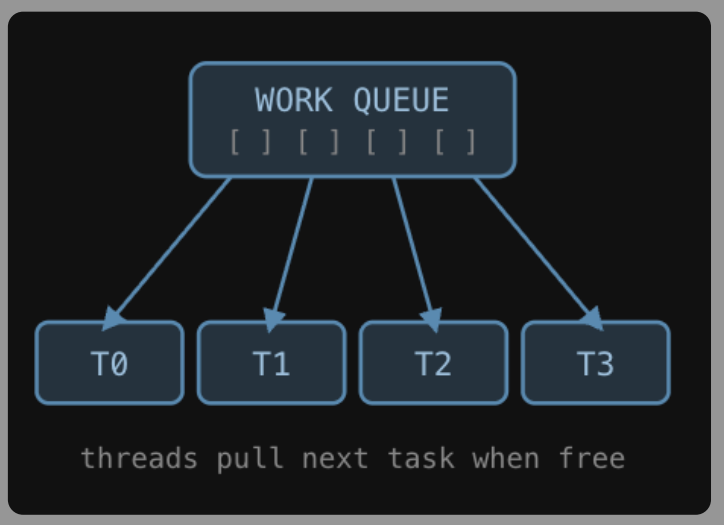
+ Work Stealing



→ Idle threads actively pull work from overloaded threads' queues
No central coordinator is needed and threads self-balance at runtime
Best when work arrival is unpredictable.



+ Dynamic Scheduling



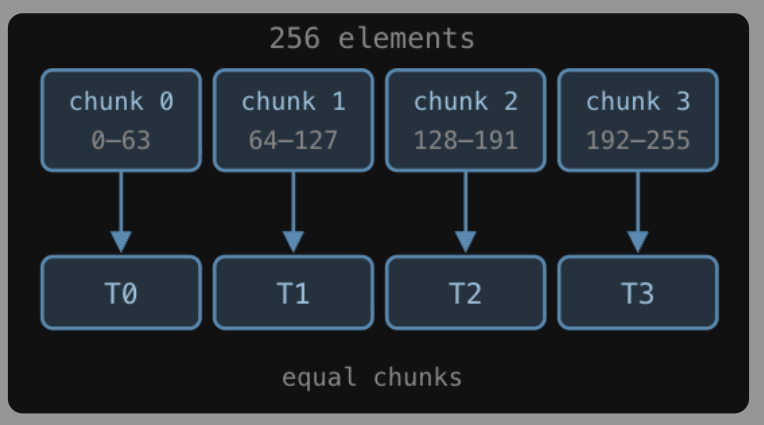
→ A central work queue distributes tasks to threads as they finish

Threads never sit idle

and

they immediately pick up the next available unit of work.

+ Chunked Iteration.



→ Divide work into equal-sized chunks assigned to threads upfront

Simple

Low-overhead

Best when work/element is roughly uniform